ABV-IIITM

Sentinel-X Intrusion Detection System

Technical Report on a Calibrated Machine Learning Pipeline for UNSW-NB15

Authors: Hrushikesh Ramilla (2023BMS023), Avadh Khandelwal (2023BMS007), Himavarshit Reddy (2023BMS018)

Course: Art of Engineering Research

Department: Engineering Sciences

Branch: Mathematics & Scientific Computing

Date: April 19, 2026

Abstract

The analyzed repository implements a complete machine-learning intrusion detection workflow for the UNSW-NB15 network security dataset. The project addresses the engineering problem of building a deployable multi-class classifier that remains reproducible, avoids target leakage, supports explainability, and exposes results through both a local API and a Streamlit dashboard. Repository evidence shows a phased development strategy in which the authors first established deterministic data governance, then built a train-only preprocessing pipeline, compared four classical machine-learning models, audited an inflated external benchmark, and finally hardened the selected production model using isotonic calibration and minority-class threshold tuning. The final production artifact is a calibrated and threshold-adjusted LightGBM model operating on 75 selected features derived from cleaned raw traffic records. Logged evaluation reports indicate an offline best score of 75.34% accuracy and 0.5346 macro F1, while the deployed production path achieved 74.73% accuracy and 0.5269 macro F1 on the untouched predefined test split. The system further includes SHAP-based explainability, a WSGI prediction API, and a browser-validated dashboard. Overall, the project demonstrates a disciplined engineering workflow in which leakage control, reproducibility, and deployment realism were prioritized over superficially higher but invalid benchmark scores.

Contents

1 Introduction **1**

2 Research Gap **2**

3 Problem Statement **3**

4 Objectives **4**

5 Methodology **5**
- 5.1 System Architecture . . . 5
- 5.2 Tools and Technologies . . . 6
- 5.3 Development Procedure . . . 8
- 5.4 Algorithm Description . . . 9

6 Results **11**
- 6.1 Dataset and Feature-Space Outcomes . . . 11
- 6.2 Comparative Model Performance . . . 11
- 6.3 Production Model Results . . . 13
- 6.4 Per-Class Behavior . . . 13
- 6.5 Development Evolution Results . . . 15
- 6.6 Leakage Audit Results . . . 16
- 6.7 Explainability and Interface Outputs . . . 17

7 Discussion / Analysis **20**

8 Conclusion **21**

9 Recommendations / Future Work **22**

10 Appendices **24**
- 10.1 Important Code Snippets . . . 24
- 10.2 Sample Development Log Excerpts . . . 25
- 10.3 Configuration Summary . . . 25
- 10.4 Artifact Inventory . . . 25

List of Figures

5.1 System Architecture reconstructed from the repository modules and pipeline logs. 6

6.1 Model metric comparison generated by the repository evaluation module. . . 12
6.2 Training-time comparison across candidate models. 13
6.3 Per-class F1 heatmap across the offline comparison models. 14
6.4 One-vs-rest ROC comparison for the candidate models. 15
6.5 Performance evolution artifact generated during final delivery packaging. . . 16
6.6 Benchmark leakage audit showing the gap between inflated and safe evaluation settings. 17
6.7 SHAP bar chart for the final production model. 18
6.8 Dashboard overview screenshot captured during automated GUI validation. . 19

List of Tables

5.1 Detected tools and technologies used in the project. 7

6.1 Final preprocessing statistics derived from `feature_report.json`. 11
6.2 Offline comparison results on the untouched predefined test split. 12
6.3 Final deployed production metrics. 13
6.4 Per-class production results from the final calibrated and thresholded model. 14
6.5 Measured production improvement between Phase 2 and Phase 3. 15
6.6 Leakage audit outcome reconstructed from `repo_benchmark_audit.md` and JSON payloads. 16

10.1 Key final configuration settings inferred from source code and generated artifacts. 25

Chapter 1

Introduction

Intrusion detection systems (IDS) remain a core defensive layer in modern networked infrastructure because malicious traffic increasingly blends with legitimate activity and must be classified under strict operational constraints. The analyzed repository, stored under the project folder name `IDS-project`, targets this problem through a supervised machine-learning pipeline for the UNSW-NB15 dataset. The codebase reveals a practical orientation: it is not limited to model training, but extends to inference services, explainability, artifact packaging, and dashboard-based human interaction.

The project operates in the domain of network intrusion detection for heterogeneous traffic categories. According to the repository README and final report artifacts, the system predicts ten traffic classes consisting of nine attack categories and one normal class. This design aligns with realistic security operations in which coarse binary alerts are insufficient for triage and downstream response.

The real-world relevance of the system arises from four conditions visible throughout the repository. First, the pipeline explicitly preserves predefined train and test partitions, indicating concern for evaluation integrity. Second, the code removes the `id` field and excludes the binary label from feature generation to prevent leakage. Third, the final artifact is exposed through a WSGI service and a Streamlit dashboard, which means the project was engineered as an operational demonstrator rather than a notebook-only exercise. Fourth, the development logs show repeated attention to minority-class performance, especially `fuzzers`, confirming that the team treated practical class imbalance as a central deployment issue.

From an engineering research perspective, the repository therefore represents a full applied workflow: data validation, preprocessing design, comparative modeling, explainability, user-facing deployment, and post-hoc audit of misleading external baselines.

Chapter 2

Research Gap

The repository provides strong evidence that the project was motivated by deficiencies in naive or public benchmark-driven IDS implementations. The most explicit gap appears in the external benchmark audit coded in `src/repo_audit.py` and documented in the generated reports. There, the team reproduced a public Random Forest benchmark that achieved 97.27% accuracy on a repository-prepared dataset, then demonstrated that this result dropped to 89.21% after removing `attack_cat_*` leakage columns. This establishes that some existing approaches report overly optimistic performance because they rely on target-derived features unavailable during real inference.

The development logs reveal three additional limitations in prior or baseline approaches.

1. Existing pipelines often optimize for raw accuracy rather than macro-level robustness across minority classes. The journal repeatedly states that macro F1, not accuracy alone, should drive model ranking because the system must remain reliable across all attack categories.

2. Existing implementations may use preprocessing decisions learned from the full dataset, which risks contamination. In contrast, the repository consistently logs that all imputers, encoders, feature selectors, and scaling operations are fit on training data only and then applied unchanged to the test split.

3. Existing academic prototypes frequently stop at static evaluation. This project instead incorporates a deployable prediction API, a dashboard, SHAP explanations, screenshot-based GUI validation, and delivery packaging.

The research gap can therefore be stated as the absence of a reproducible, deployment-aware IDS pipeline that combines leakage-free data preparation, transparent comparative evaluation, calibrated inference, minority-class handling, and explainable operational interfaces. The improvement introduced by this project lies not in proposing a novel learning theory, but in constructing an end-to-end engineering workflow that makes benchmark claims more honest and deployment behavior more realistic.

Chapter 3

Problem Statement

The engineering challenge addressed by the repository is to design and implement a multi-class intrusion detection system for the UNSW-NB15 dataset that can classify raw network traffic into ten traffic categories while satisfying four constraints: leakage-free preprocessing, bounded computational complexity, reproducible evaluation on a predefined test split, and deployable inference through machine-readable and human-readable interfaces.

More precisely, the system must ingest raw CSV datasets, validate schema consistency, transform mixed numerical and categorical features, remove spurious identifiers and redundant attributes, train multiple candidate models under the same experimental protocol, select a production model using metrics that reflect class imbalance, generate interpretable predictions, and expose the final artifact through both an API and a dashboard. The logs also indicate a performance target of exceeding 0.60 macro F1 for the production system, a goal that was not achieved but that guided later optimization phases.

The required outputs are therefore:

- a reproducible training pipeline,
- serialized preprocessing and model artifacts,
- comparative evaluation reports and plots,
- a deployable prediction and explanation service,
- a dashboard for inspection and demonstration,
- and formal delivery materials suitable for academic review.

Chapter 4

Objectives

Based on implementation details and logged upgrade targets, the project objectives can be reconstructed as follows:

1. Build a leakage-free preprocessing pipeline that removes identifier fields, prevents target contamination, and applies all learned transformations using train-only fitting.
2. Train and compare at least four candidate classifiers under a common three-fold stratified cross-validation regime and a fixed predefined train/test split.
3. Achieve production-quality multi-class performance using macro F1 as the primary selection criterion, while preserving competitive accuracy on the untouched test set.
4. Improve minority-class recovery, especially for `fuzzers`, through controlled imbalance mitigation, calibration, and threshold tuning.
5. Provide reproducible explainability outputs using SHAP and integrate them into user-facing system components.
6. Deliver a working local deployment stack consisting of serialized artifacts, a prediction API, and a Streamlit dashboard validated through automated GUI checks.

Chapter 5

Methodology

5.1 System Architecture

The repository follows a layered architecture centered on a single orchestration script, `main.py`. Data are first read and validated by `src/data_io.py`. They are then cleaned, encoded, pruned, and transformed by `src/preprocess.py`. Model construction, class balancing, calibration, and threshold tuning are performed by `src/train.py`. Evaluation and visualization are produced by `src/evaluate.py`. Production inference is handled by `src/inference.py`, while `src/explain.py` generates SHAP artifacts. Finally, `app.py` exposes a WSGI API and `dashboard.py` provides a Streamlit interface using `src/dashboard_service.py` as an adapter layer.

This structure is noteworthy because it separates offline experimental comparison from deployment logic. The logs repeatedly distinguish between an "offline comparison winner" and a "production final model." That separation is enforced in code by training all models only in `train_all` mode, then calibrating only the selected winner for deployment.

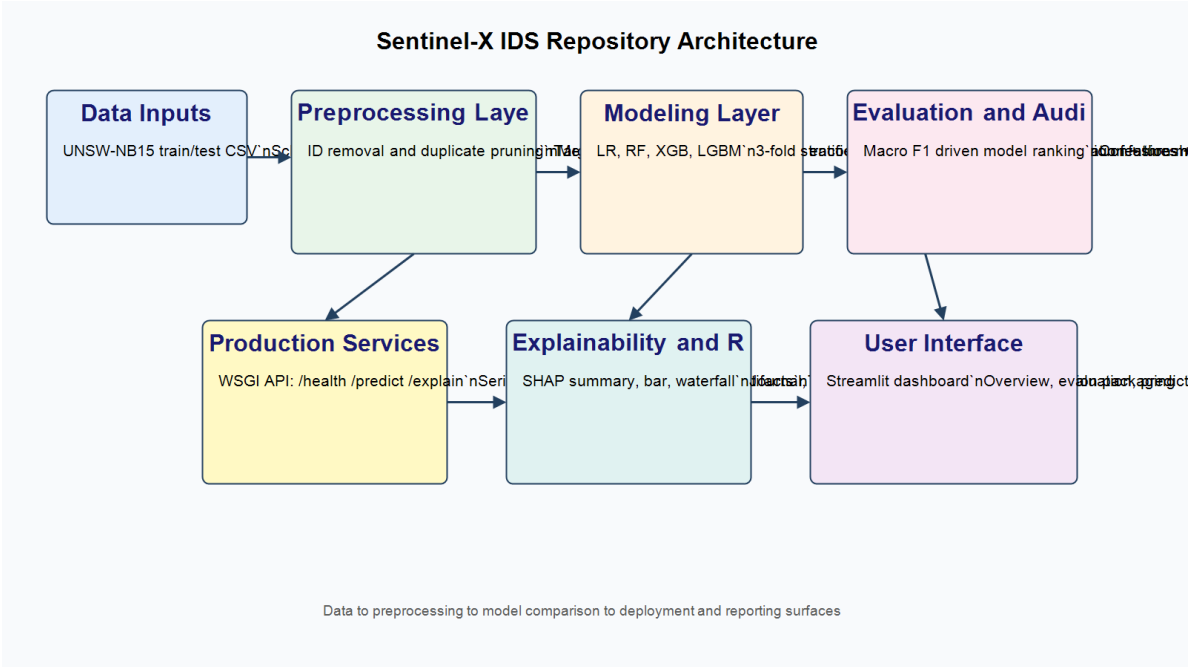

Figure 5.1: System Architecture reconstructed from the repository modules and pipeline logs.

5.2 Tools and Technologies

The technology stack was inferred from `requirements.txt`, the source code, and the generated artifacts.

Category	Tool / Library	Observed role in the repository
Programming language	Python 3.11	Primary implementation language for training, inference, reporting, and UI
Data processing	pandas, numpy, pyarrow	CSV ingestion, tabular transformation, numeric handling, and parquet caching
Machine learning	scikit-learn	preprocessing pipelines, feature selection, scaling, logistic regression, metrics, calibration
Ensemble models	RandomForest, XGBoost, LightGBM	four-model comparison and final production model selection
Imbalance handling	imbalanced-learn (SMOTE)	controlled resampling applied to train-only subsets
Explainability	SHAP	summary, bar, and waterfall explanations for the final model
Visualization	matplotlib, seaborn, Plotly/Streamlit rendering	report plots, heatmaps, ROC curves, dashboard presentation
Interface layer	WSGI + Streamlit	API endpoints plus interactive dashboard pages
Serialization	joblib, JSON, Markdown	storage of models, metadata, metrics, and run history
Testing / automation	Playwright, browser automation scripts	reproducible dashboard validation and

5.3 Development Procedure

The project journal in `artifacts/logs/project_journal.md` provides unusually strong chronological evidence, allowing reconstruction of the development procedure as an engineering timeline rather than as a guessed narrative.

Phase 1: Setup and Run Governance

The pipeline first created standardized directories, initialized the project journal and run summary, and captured environment metadata. This indicates that reproducibility and artifact management were treated as first-order requirements.

Phase 2: Data Ingestion and Profiling

The system loaded the predefined UNSW-NB15 training and testing CSV files, validated that schemas matched, checked for the presence of the target field, and attempted parquet caching to accelerate repeated runs. This phase also produced label-distribution and missing-value plots.

Phase 3 and Phase 5: Cleaning, Preprocessing, and Feature Reduction

The code replaced infinite values, removed exact duplicates after dropping `id`, encoded categorical fields, pruned highly correlated numeric features above a fixed threshold of 0.95, generated interaction features, applied univariate feature selection, and scaled selected features for the logistic-regression branch.

Phase 4: Decision-Focused Exploratory Data Analysis

Rather than produce a large descriptive notebook, the repository intentionally restricted EDA to decision-relevant plots: missingness, class balance, numeric distributions, boxplots, correlations, and categorical frequencies.

Phase 6 and Phase 7: Training and Comparison

Four bounded-complexity models were trained: Logistic Regression, Random Forest, XGBoost, and LightGBM. Cross-validation was executed with `StratifiedKFold` using three folds. Later iterations added controlled SMOTE and XGBoost sample weights on train-only subsets.

Phase 8: Final Evaluation and Selection

The models were evaluated on the untouched predefined test split using accuracy, macro F1, weighted F1, ROC-AUC, confusion matrices, and macro false alarm rate. LightGBM was consistently selected as the winning offline model.

Phase 8B: Benchmark Leakage Audit

The repository compared a public benchmark against a leakage-free variant, proving that inflated public scores depended on `attack_cat`-derived dummy columns.

Phase 9: Explainability

The final production model received bounded-cost SHAP analysis using a stratified sample. Summary, bar, and waterfall plots were generated and persisted.

Phase 10 and Phase 11: Controlled Upgrades and Final Packaging

The development evidence shows two significant optimization cycles. Phase 2 increased feature capacity to 50, added train-only SMOTE, applied class weighting and XGBoost sample weights, and introduced isotonic calibration. Phase 3 increased the feature cap to 75, introduced interaction features, reduced SMOTE aggressiveness to 30% of majority-class size, and tuned thresholds for selected minority classes. The final stage rebuilt the project from a cleaned artifact state, validated the GUI, captured screenshots, and generated presentation assets.

5.4 Algorithm Description

The system is not dominated by a single novel algorithm; instead, its contribution lies in the controlled orchestration of preprocessing, model comparison, calibration, and threshold-aware inference. The production procedure inferred from `main.py`, `train.py`, and `inference.py` is summarized below.

Algorithm 1 Repository-derived production training and inference workflow

```
Load predefined training and testing CSV files
Validate schema consistency and target availability
Remove id, normalize labels, replace infinite values, and drop duplicates
Create interaction features when source columns exist
Split predictors into numeric and categorical sets
Fit train-only imputers and one-hot encoders
Remove highly correlated numeric variables
Apply univariate feature selection with cap k = min(feature_cap, d)
Scale selected features for the logistic-regression branch
for each candidate model in {LR, RF, XGB, LGBM} do
    Perform three-fold stratified cross-validation on training data
    Apply controlled SMOTE to train-only subsets
    Fit the estimator and record training time
end for
Evaluate all candidates on the untouched test split
Select the best model using macro F1, then accuracy, then inverse macro FAR
Retrain only the selected model as calibrated production artifact
Hold out a validation subset and tune minority-class probability thresholds
Save model, preprocessing artifacts, metrics, SHAP plots, and manifest files
During inference, transform raw records, apply calibrated probabilities, then override
argmax using tuned class thresholds when applicable
```

The dominant computational costs are in preprocessing transforms, repeated model fits during cross-validation, SMOTE expansion, and SHAP computation. Complexity depends on model family and data dimensionality, but the repository logs show that later optimization phases notably increased training time because of resampling and calibration rather than because of user-interface overhead.

Chapter 6

Results

6.1 Dataset and Feature-Space Outcomes

The final feature report provides measurable preprocessing outcomes. After dropping `id` and exact duplicates, the training set reduced from 82,332 to 55,945 rows and the test set from 175,341 to 107,740 rows. Seven correlated features were removed in the final run, and the model operated on 75 selected features drawn from an original transformed space of 186 features. The final class set comprised ten labels: analysis, backdoor, dos, exploits, fuzzers, generic, normal, reconnaissance, shellcode, and worms.

Quantity	Training split	Testing split
Rows before cleaning	82,332	175,341
Rows after duplicate removal	55,945	107,740
Duplicates removed	26,387	67,601
Features after selection	75	
Classes modeled	10	

Table 6.1: Final preprocessing statistics derived from `feature_report.json`.

6.2 Comparative Model Performance

The main evaluation artifact, `model_metrics.json`, shows that LightGBM achieved the best offline macro F1 and was therefore selected as the comparison winner. XGBoost was close in macro F1 and obtained the best raw offline accuracy, but LightGBM offered the best ranking outcome under the project's macro-F1-first criterion.

Model	Accuracy	Macro F1	Weighted F1	ROC-AUC (OvR)
Logistic Regression	68.60%	0.4307	0.7087	0.9194
Random Forest	74.48%	0.5032	0.7171	0.9467
XGBoost	75.81%	0.5307	0.7434	0.9586
LightGBM	75.34%	0.5346	0.7244	0.9529

Table 6.2: Offline comparison results on the untouched predefined test split.

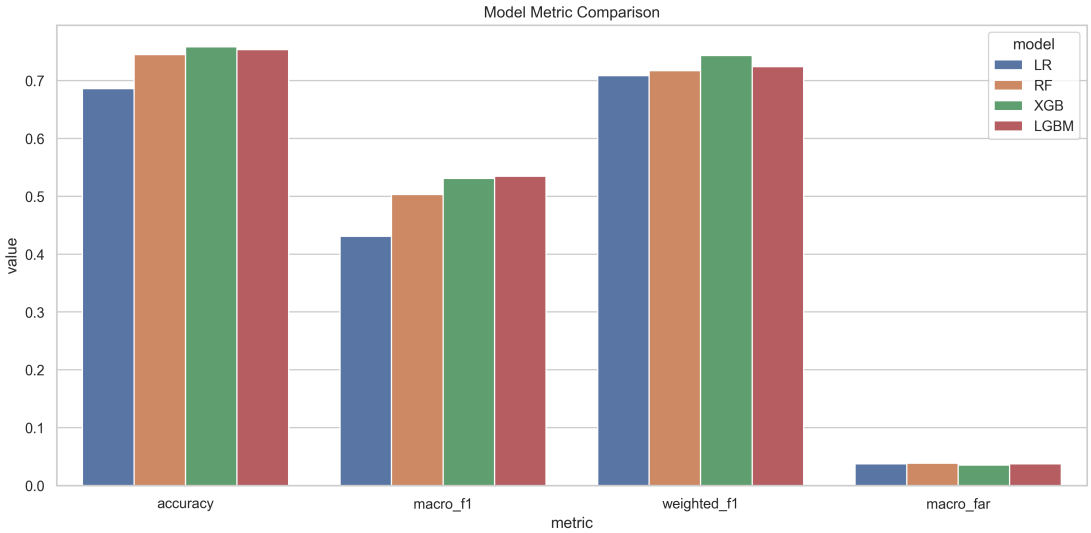

Figure 6.1: Model metric comparison generated by the repository evaluation module.

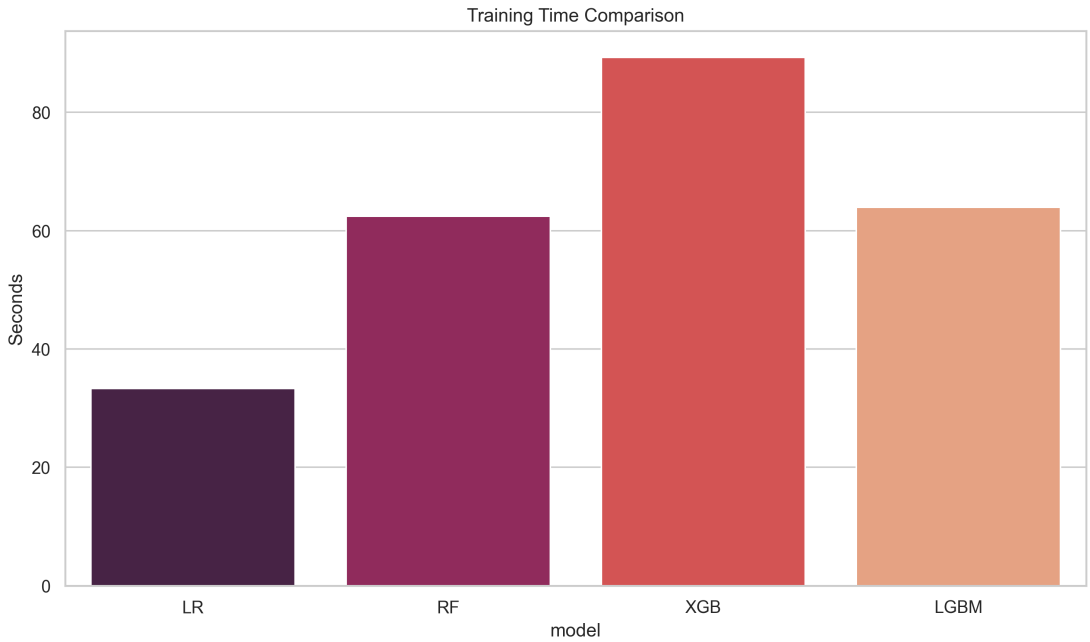

Figure 6.2: Training-time comparison across candidate models.

6.3 Production Model Results

The deployed model is not the raw offline LightGBM estimator. Instead, the repository trains a calibrated production artifact using isotonic calibration and then tunes class-specific thresholds for `fuzzers`, `backdoor`, `analysis`, and `worms`. The final production metrics recorded in `final_summary.json` are shown in Table 6.3.

Metric	Production value
Final model	Calibrated + thresholded LightGBM
Accuracy	74.73%
Macro F1	0.5269
Weighted F1	0.7125
Threshold tuning validation macro F1	0.5845

Table 6.3: Final deployed production metrics.

6.4 Per-Class Behavior

The class-level report shows a common IDS pattern: dominant classes such as `normal`, `generic`, and `exploits` are modeled much more strongly than minority attack categories. Nevertheless, the Phase 3 upgrade improved the `fuzzers` class materially.

Class	Precision	Recall	F1-score
analysis	0.1601	0.2848	0.2050
backdoor	0.2338	0.2111	0.2218
dos	0.2873	0.2425	0.2630
exploits	0.8251	0.8019	0.8133
fuzzers	0.7076	0.1557	0.2552
generic	0.8617	0.8512	0.8565
normal	0.7752	0.9791	0.8653
reconnaissance	0.7902	0.7210	0.7540
shellcode	0.6943	0.4830	0.5697
worms	0.4023	0.5512	0.4651

Table 6.4: Per-class production results from the final calibrated and thresholded model.

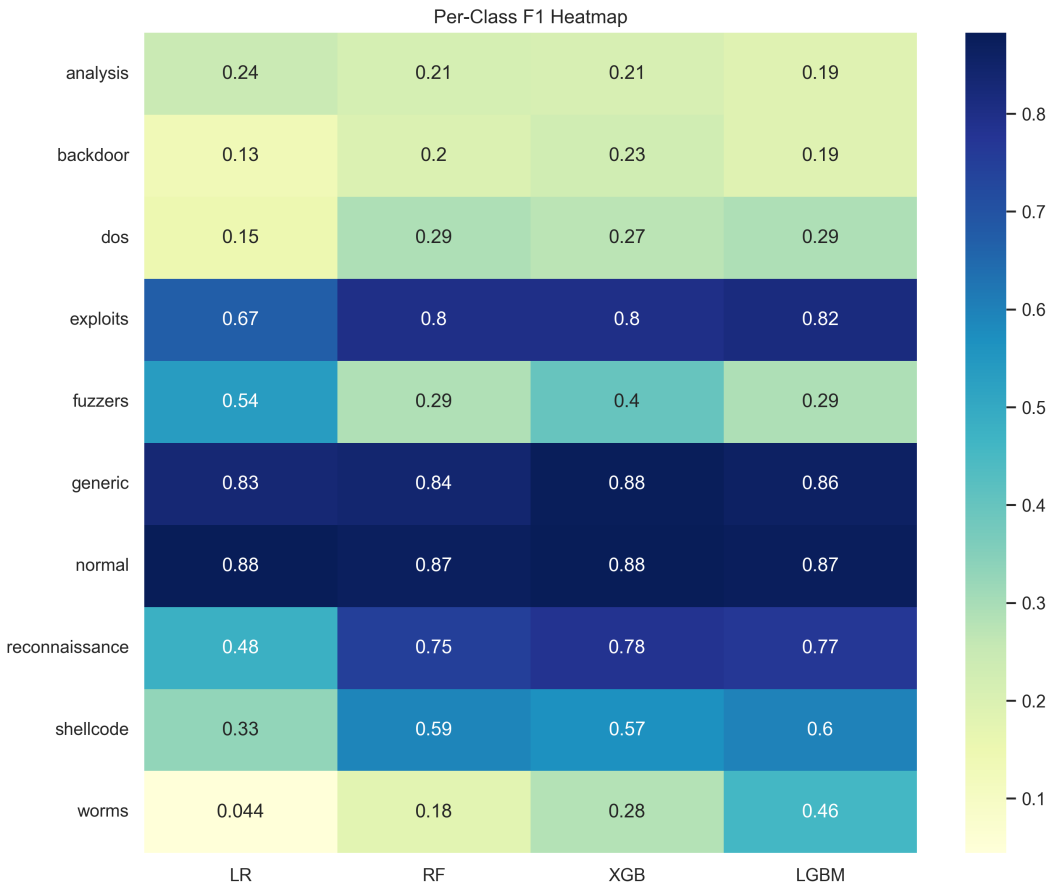

Figure 6.3: Per-class F1 heatmap across the offline comparison models.

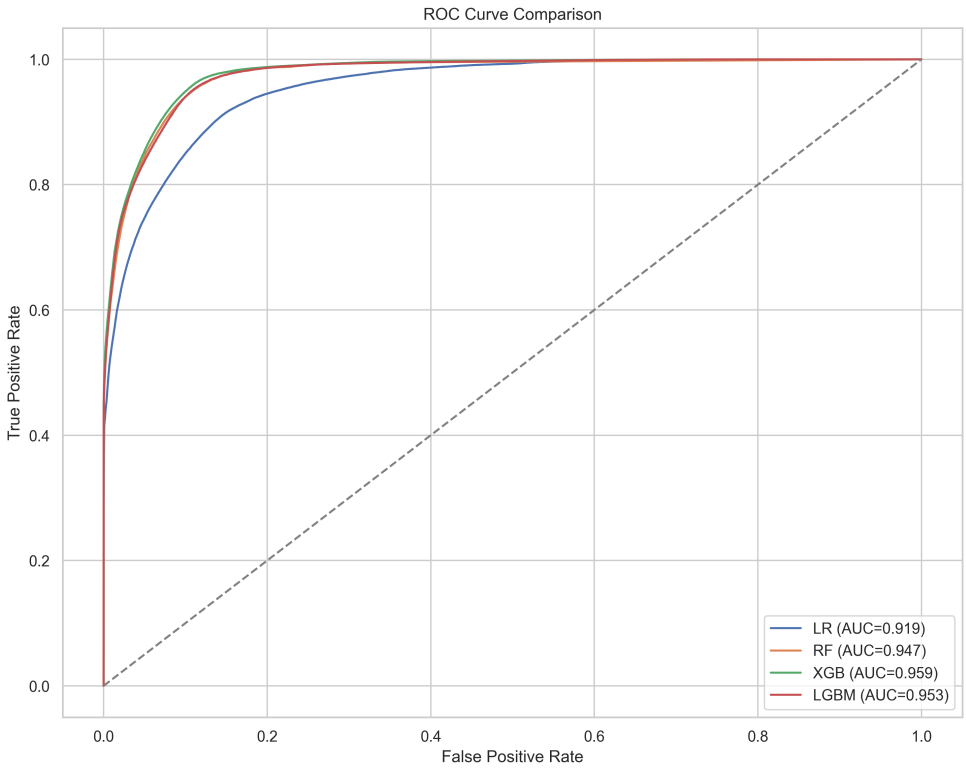

Figure 6.4: One-vs-rest ROC comparison for the candidate models.

6.5 Development Evolution Results

The repository explicitly tracked two controlled optimization cycles. Table 6.5 summarizes the measured improvement between the logged Phase 2 and Phase 3 production baselines.

Metric	Phase 2 production	Phase 3 production	Change
Accuracy	73.49%	74.73%	+1.24 points
Macro F1	0.4992	0.5269	+0.0277
Weighted F1	0.6890	0.7125	+0.0235
Fuzzers recall	0.0777	0.1557	+0.0780
Fuzzers F1	0.1390	0.2552	+0.1162

Table 6.5: Measured production improvement between Phase 2 and Phase 3.

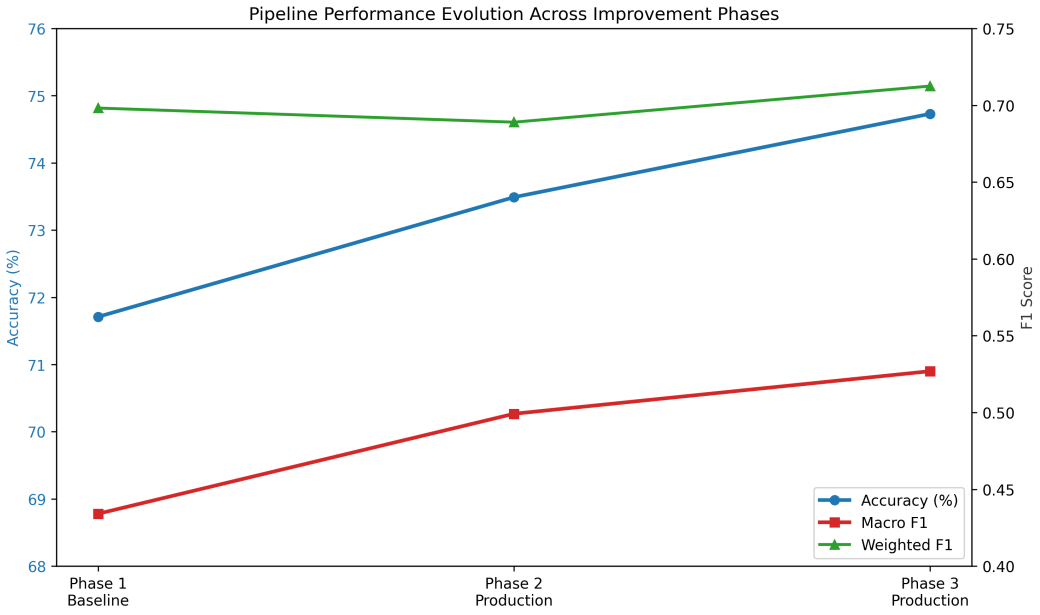

Figure 6.5: Performance evolution artifact generated during final delivery packaging.

6.6 Leakage Audit Results

One of the most important empirical findings in the repository is the gap between a leakage-inflated public benchmark and the safe benchmark obtained after removing target-derived columns.

Scenario	Accuracy	Macro F1
Public repo style with leakage	97.27%	not the production reference path
Same benchmark without `attack_cat_*` leakage columns	89.21%	lower but operationally valid

Table 6.6: Leakage audit outcome reconstructed from `repo_benchmark_audit.md` and JSON payloads.

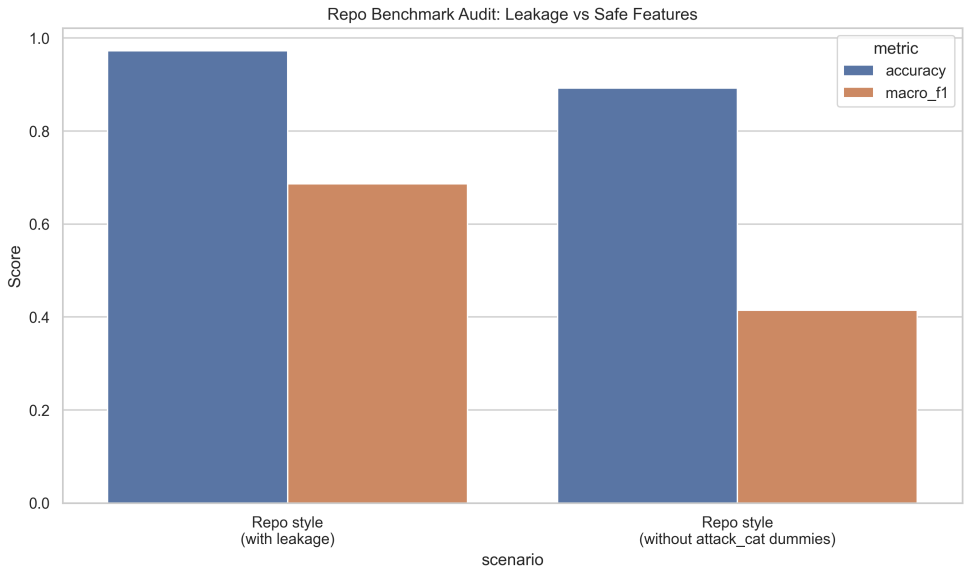

Figure 6.6: Benchmark leakage audit showing the gap between inflated and safe evaluation settings.

6.7 Explainability and Interface Outputs

The project generated SHAP summary, bar, and waterfall plots for the final production model, and the final delivery report confirms successful GUI validation for dashboard startup, manual prediction, CSV upload with `id`, and explainability display.

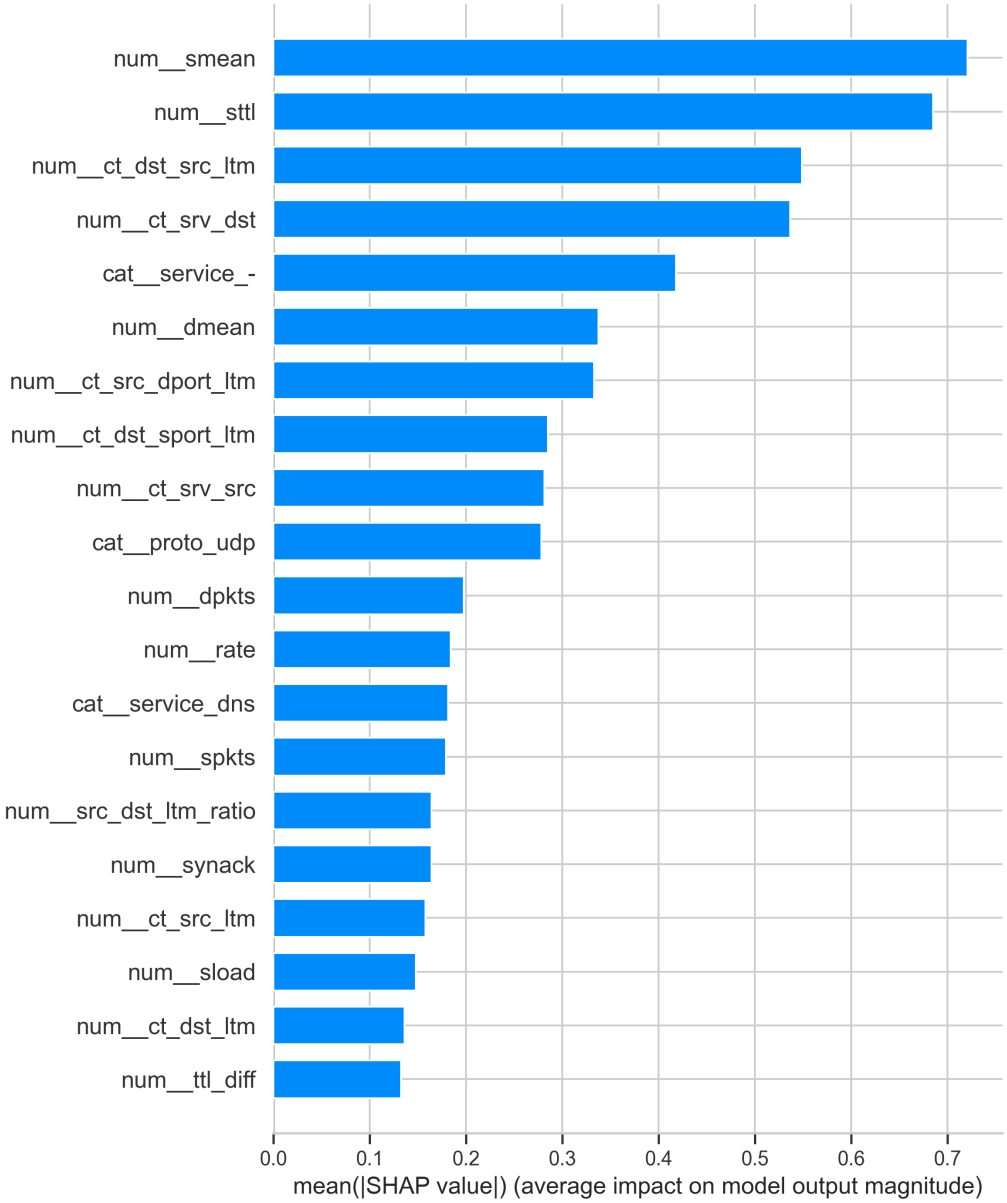

Figure 6.7: SHAP bar chart for the final production model.

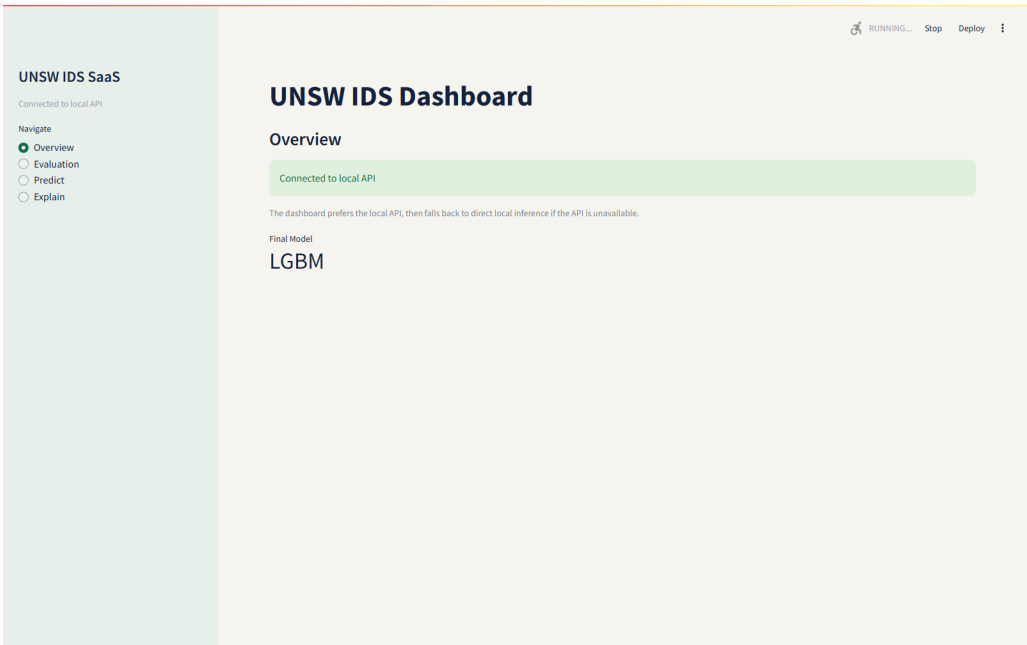

Figure 6.8: Dashboard overview screenshot captured during automated GUI validation.

Chapter 7

Discussion / Analysis

The results indicate that the repository achieved its central engineering goal: it produced a realistic, deployment-aware IDS pipeline with measurable multi-class performance and strong reproducibility controls. The most important reason the final outcomes are credible is that the project deliberately constrained itself to leakage-free raw features and a fixed train/test split. This design choice reduces headline accuracy relative to inflated public benchmarks, but it strengthens scientific validity.

The comparative results also justify the model selection logic. Although XGBoost attained slightly higher offline accuracy, LightGBM obtained the highest macro F1 and was therefore a better fit for the stated objective of balanced multi-class detection. This is consistent with the repository's repeated emphasis on minority-class robustness rather than majority-class dominance.

The Phase 2 and Phase 3 logs reveal a disciplined optimization strategy. Rather than repeatedly rewriting the architecture, the team performed controlled interventions: increasing feature capacity, adding interaction features, introducing train-only SMOTE, calibrating only the deployable artifact, and tuning class thresholds for specific under-served categories. This is good engineering practice because it permits causal interpretation of improvements. The measured rise in production macro F1 from 0.4992 to 0.5269 supports the claim that these interventions improved operational behavior.

However, the remaining limitations are also clear. Minority-class recall remains weak, especially for `fuzzers`. The final production model has high precision for that class but low recall, which means the classifier is still conservative in recovering those attacks. The logs explicitly state that the target of 0.60+ macro F1 was not reached. This shortfall likely arises from a combination of class imbalance, partial class overlap in the UNSW-NB15 feature space, and the bounded complexity of the chosen classical model family.

The project also exposes a useful methodological lesson. Calibration and threshold tuning improved deployment realism but slightly changed the relationship between offline comparison metrics and production behavior. That divergence is not a defect; it is evidence that the repository correctly separates benchmarking from deployed inference policy.

Overall, the system substantially closes the research gap identified earlier: it replaces inflated benchmark chasing with a reproducible, auditable, explainable, and deployable IDS engineering workflow.

Chapter 8

Conclusion

The analyzed repository implements a complete intrusion detection engineering pipeline for the UNSW-NB15 dataset. It successfully integrates schema validation, leakage-aware preprocessing, comparative model training, calibrated production inference, explainability, and user-facing deployment components. The final deployable artifact is a calibrated and threshold-tuned LightGBM model operating on 75 selected features and evaluated on an untouched predefined test set.

The final production system achieved 74.73% accuracy and 0.5269 macro F1, while the offline comparison winner achieved 75.34% accuracy and 0.5346 macro F1. These results do not satisfy the aspirational 0.60+ macro F1 target, but they represent a meaningful and well-documented improvement over earlier baselines. Most importantly, the repository demonstrates system readiness in the sense relevant to engineering coursework: the pipeline is modular, auditable, reproducible, and deployable.

Chapter 9

Recommendations / Future Work

Several future improvements are strongly suggested by the repository's current strengths and remaining gaps.

1. Introduce more advanced class-sensitive optimization, such as focal loss variants or cost-sensitive boosting, to improve recovery for minority attack classes without excessive oversampling.

2. Replace the small threshold grid with a more systematic search strategy or Bayesian optimization over class-specific decision thresholds.

3. Expand feature engineering beyond the three current interaction terms using domain-guided traffic ratios, temporal burst descriptors, or protocol-specific aggregations.

4. Add formal unit and regression tests for preprocessing invariants, especially around schema drift, missing columns, and feature-order consistency between training and inference.

5. Extend monitoring artifacts so runtime prediction distributions, confidence drift, and API error conditions can be logged continuously in a production-like environment.

6. Investigate sequence-aware or hybrid models only after preserving the repository's current anti-leakage and deployment discipline, so any accuracy gains remain operationally credible.

References

[1] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer. Smote: Synthetic minority over-sampling technique, 2002. Class-imbalance mitigation concept applied in train-only subsets.

[2] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system, 2016. Boosted-tree model family used in the comparison pipeline.

[3] Guolin Ke et al. Lightgbm: A highly efficient gradient boosting decision tree, 2017. Winning offline comparison model and deployed production model family.

[4] Scott Lundberg and Su-In Lee. A unified approach to interpreting model predictions, 2017. Explainability method used for summary, bar, and waterfall plots.

[5] Nour Moustafa and Jill Slay. Unsw-nb15: A comprehensive data set for network intrusion detection systems, 2015. Dataset and supporting publication referenced by the repository design.

[6] Fabian Pedregosa et al. Scikit-learn: Machine learning in python, 2011. Core preprocessing, model evaluation, calibration, and baseline estimator toolkit.

[7] Streamlit Inc. Streamlit documentation, 2026. Dashboard framework used for interactive project presentation.

Chapter 10

Appendices

10.1 Important Code Snippets

The following repository excerpts are representative of the engineering decisions identified in the analysis.

Snippet A: Interaction feature engineering

```
if "sload" in engineered.columns and "dload" in engineered.columns:
    engineered["sload_dload_ratio"] = engineered["sload"] / (engineered["dload"] + 1e-6)

if "ct_src_ltm" in engineered.columns and "ct_dst_ltm" in engineered.columns:
    engineered["src_dst_ltm_ratio"] = engineered["ct_src_ltm"] / (engineered["ct_dst_ltm

if "sttl" in engineered.columns and "dttl" in engineered.columns:
    engineered["ttl_diff"] = engineered["sttl"] - engineered["dttl"]
```

Snippet B: Controlled SMOTE strategy

```
smote_ratio = 0.3
majority_count = max(distribution.values())
target_count = max(1, int(np.ceil(majority_count * smote_ratio)))
strategy = {cls: target_count for cls, count in distribution.items() if count < target_c
```

Snippet C: Threshold-aware production inference

```
if probabilities is not None:
    predicted_indices = _thresholded_predictions(
        probabilities,
        class_thresholds,
        list(artifacts.label_encoder.classes_),
    )
else:
```

```
predicted_indices = artifacts.final_model.predict(X_infer)
```

10.2 Sample Development Log Excerpts

```
Phase 8B - External Repo Benchmark Audit
- Repo-style benchmark accuracy with leakage: 97.27%
- Repo-style benchmark accuracy without attack_cat leakage columns: 89.21%

Phase 10 - Final Delivery Packaging
- Offline best-model metrics: accuracy 75.34%, macro F1 0.5346
- Production metrics: accuracy 74.73%, macro F1 0.5269
- GUI validation passed, including CSV upload containing id
```

10.3 Configuration Summary

Configuration key	Final observed value
Target column	`attack_cat`
Binary reference column	`label`
Correlation threshold	0.95
Feature cap	75
SHAP sample size configuration	300 (effective bounded sample smaller in code)
Cross-validation folds	3
Default production model	`lgbm`
Production thresholds	`fuzzers=0.30`, `backdoor=0.15`, `analysis=0.20`, `worms=0.15`

Table 10.1: Key final configuration settings inferred from source code and generated artifacts.

10.4 Artifact Inventory

Artifact	Purpose
`artifacts/reports/model_metrics.json`	Structured comparison metrics for all candidate models
`artifacts/reports/final_summary.json`	Final production-model metadata, class thresholds, and sample outputs
`artifacts/reports/feature_report.json`	Cleaning statistics, dropped correlations, class mapping, and selected features
`artifacts/reports/repo_benchmark_audit.md`	Narrative explanation of leakage inflation in the external benchmark
`artifacts/logs/project_journal.md`	Chronological engineering history across pipeline phases

Artifact	Purpose
`artifacts/plots/*.png`	Evaluation, SHAP, and benchmark audit visual evidence
`artifacts/screenshots/*.png`	Browser-validated GUI evidence for the dashboard
`models/*.pkl`	Serialized preprocessing objects and trained estimators